



Introduction to Streaming

LECTURE

Aggregations, Time Windows, Watermarks



Agenda topics

Aggregations, Time Windows, Watermarks

Aggregations, Time Windows, Watermarks

Event Time + Aggregations Over Time Windows

Time Windows Aggregation with Watermark (D)

Streaming Joins

Streaming Joins

Stream/Stream Joins (D)



Types of Stream Processing

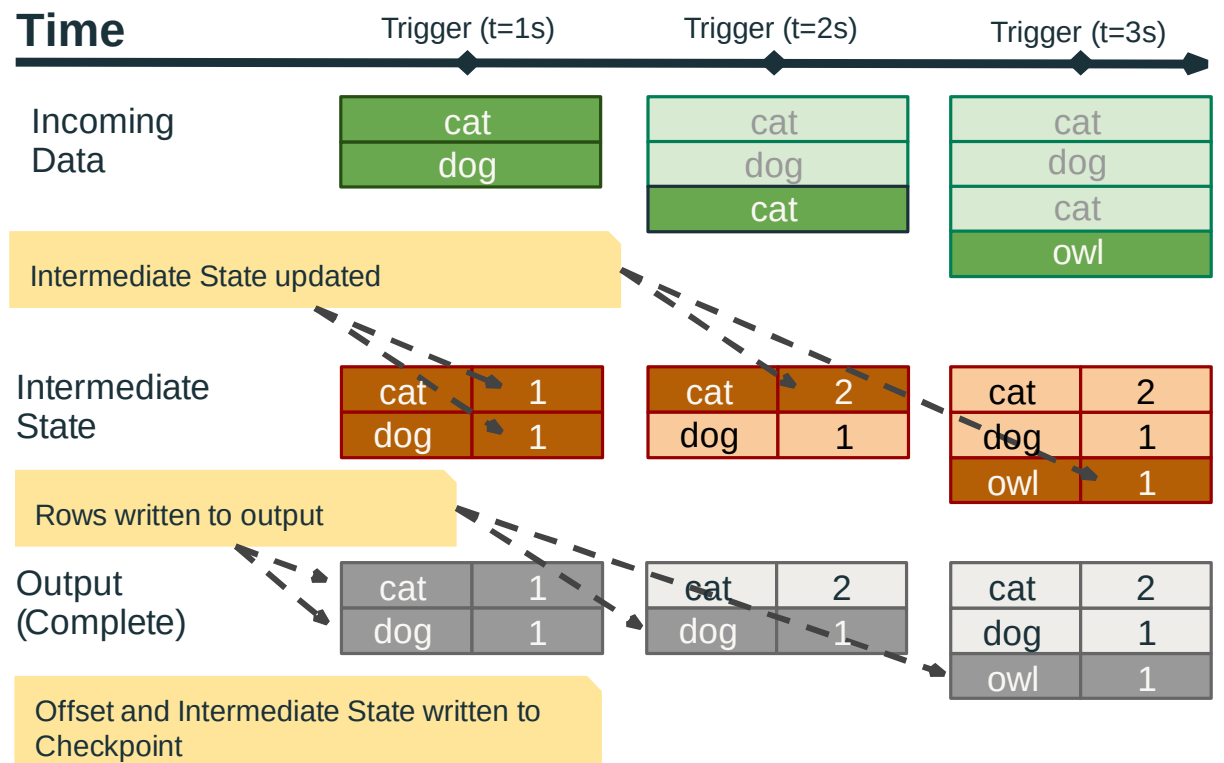
Stateless vs. Stateful processing

- **Stateless**
 - Typically trivial transformations. The way records are handled do not depend on previously seen records.
 - Example: Data Ingest (map-only), simple dimensional joins
- **Stateful**
 - Previously seen records can influence new records
 - Example: Aggregations over time, Fraud/Anomaly Detection



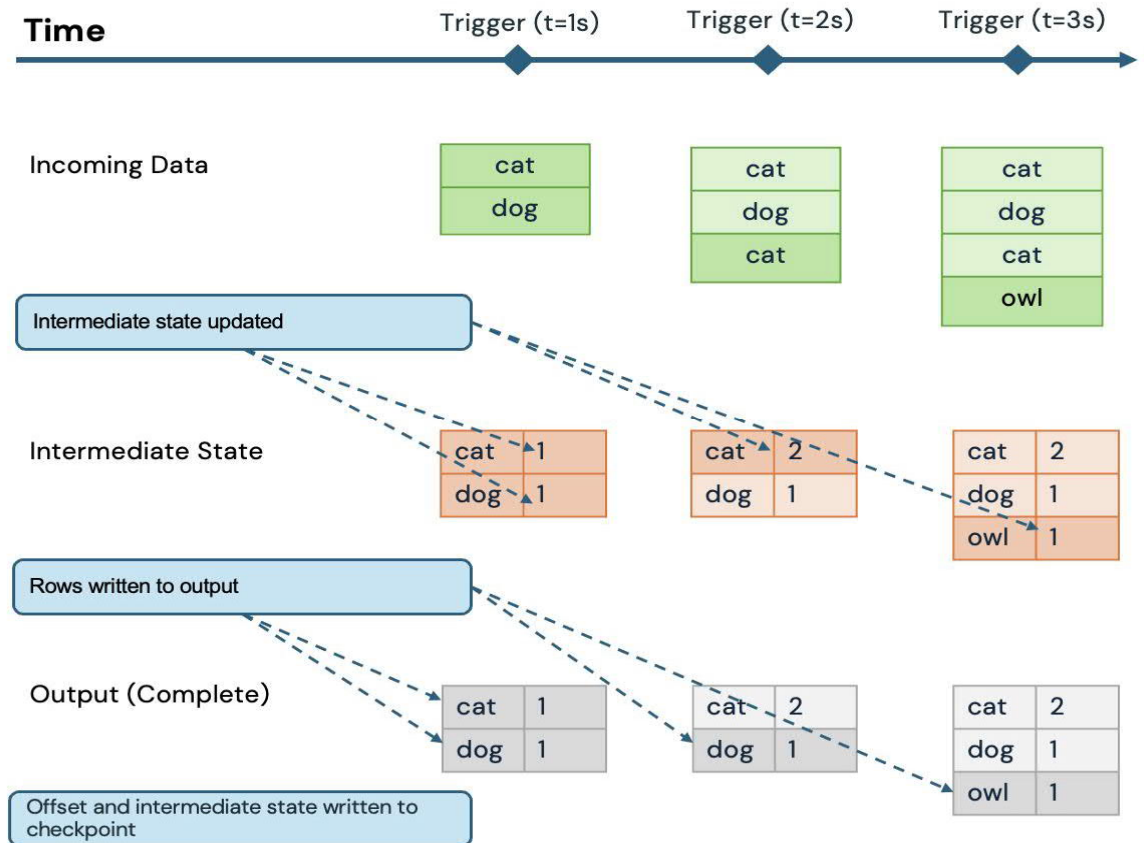
Stateful Stream

```
spark
  .readStream
  .<source info>
  .groupBy(animal)
  .count()
  .writeStream
  .mode("complete")
  .<sink info>
  .trigger("1s")
  .start()
```



Stateful Stream

```
spark
.readStream
.<source info>
.groupBy(animal)
.count()
.writeStream
.mode("complete")
.<sink info>
.trigger("1s")
.start()
```





Introduction to Streaming

LECTURE

Event Time + Aggregations over Time Windows



Reasoning About Time

Event time vs. Processing time

Event time vs. processing time

- **Event Time:** time at which the event (record in the data) actually occurred.
- **Processing time:** time at which a record is actually processed.
- Important in every use case processing unbounded data in whatever order (otherwise no guarantee on correctness)



Time Based Windows

Tumbling window vs. Sliding window

Tumbling Window

- **No window overlap**
- Any given event gets aggregated into **only one** window group (e.g. 1:00–2:00 am, 2:00–3:00 am, 3:00–4:00 am, ...)

Sliding Window

- **Windows overlap**
- Any given event gets aggregated into **multiple window** groups (e.g. 1:00–2:00 am, 1:30–2:30 am, 2:00–3:00 am, ...)



Comparison Table

Feature	Tumbling Window	Sliding Window
Overlap	None (Discrete intervals)	Yes (Overlapping intervals)
Parameters	<code>windowDuration</code>	<code>windowDuration</code> , <code>slideDuration</code>
Data Count	Each event belongs to one window	Each event can belong to multiple windows
Visual Logic	<code>[0-10]</code> , <code>[10-20]</code> , <code>[20-30]</code>	<code>[0-10]</code> , <code>[5-15]</code> , <code>[10-20]</code>
Primary Goal	Segmenting time into blocks	Tracking trends over time

Sliding windows are fixed-size but can overlap. You define the window duration (how long the window is) and the sliding interval (how often the window starts).

window Function

spark function that generates tumbling, sliding or delayed stream time

```
window(  
    timeColumn: Column,  
    windowDuration: String):  
Column (1)
```

*Creates a tumbling time window
with slideDuration as windowDuration and 0 second for startTime*

```
window(  
    timeColumn: Column,  
    windowDuration: String,  
    slideDuration: String):  
Column (2)
```

Creates a sliding time window with 0 second for startTime

```
window(  
    timeColumn: Column,  
    windowDuration: String,  
    slideDuration: String,
```

Creates a delayed time window

1) Tumbling window (non-overlapping)

Each record falls into exactly one window.

```
1 from pyspark.sql.functions import col, window
2
3 # Count events per 10-minute tumbling window
4 agg = (
5     df
6     .withWatermark("event_time", "15 minutes")      # recommended for state cleanup
7     .groupBy(window(col("event_time"), "10 minutes")) # tumbling: single duration
8     .count()
9 )
10
```

- Using only windowDuration (e.g., "10 minutes") produces **tumbling** windows.
- withWatermark is strongly recommended for windowed aggregations to bound state and handle late data. [apache +1](#)

2) Sliding window (overlapping)

A record may contribute to **multiple** windows (e.g., every 5 minutes over the last 10).

```
1 from pyspark.sql.functions import col, window
2
3 # 10-minute window that slides every 5 minutes
4 agg = (
5     df
6     .withWatermark("event_time", "15 minutes")
7     .groupBy(window(col("event_time"), "10 minutes", "5 minutes")) # sliding: duration +
8     .count()
9 )
```

- When `slideDuration` is **smaller** than `windowDuration`, windows **overlap**, so the same event appears in several windows.
- This is useful for rolling KPIs (moving averages, rolling counts). apache +1

3) (Optional) Window alignment / offset

If you want windows aligned to an offset (e.g., start at 00:02 instead of 00:00), pass `startTime`:

    `</>` Python

```
1 # Tumbling windows of 10 minutes, aligned to an offset (e.g., "2 minutes")
2 agg = df.groupby(window(col("event_time"), "10 minutes", "10 minutes", "2
    minutes")).count()
```

- When `windowDuration == slideDuration`, this is still **tumbling** (just offset).
- The 4-arg overload is documented in the API/internals references. [gitbooks](#)

```
00:00-00:10, 00:10-00:20, 00:20-00:30 ...
```

```
1 window("event_time", "10 minutes", "10 minutes", "2 minutes")
```

```
00:02-00:12
```

```
00:12-00:22
```

```
00:22-00:32
```

```
...
```

✗ Normal tumbling (no offset)

|----10min----|----10min----|----10min----|
00:00 00:10 00:20 00:30

✓ Offset tumbling (offset = 2 minutes)

|----10min----|----10min----|----10min----|
00:02 00:12 00:22 00:32

Don't forget watermarking!

When using windows, you should always define a **Watermark**. This tells Spark how long to wait for "late data" before closing a window and clearing it from memory.

Python

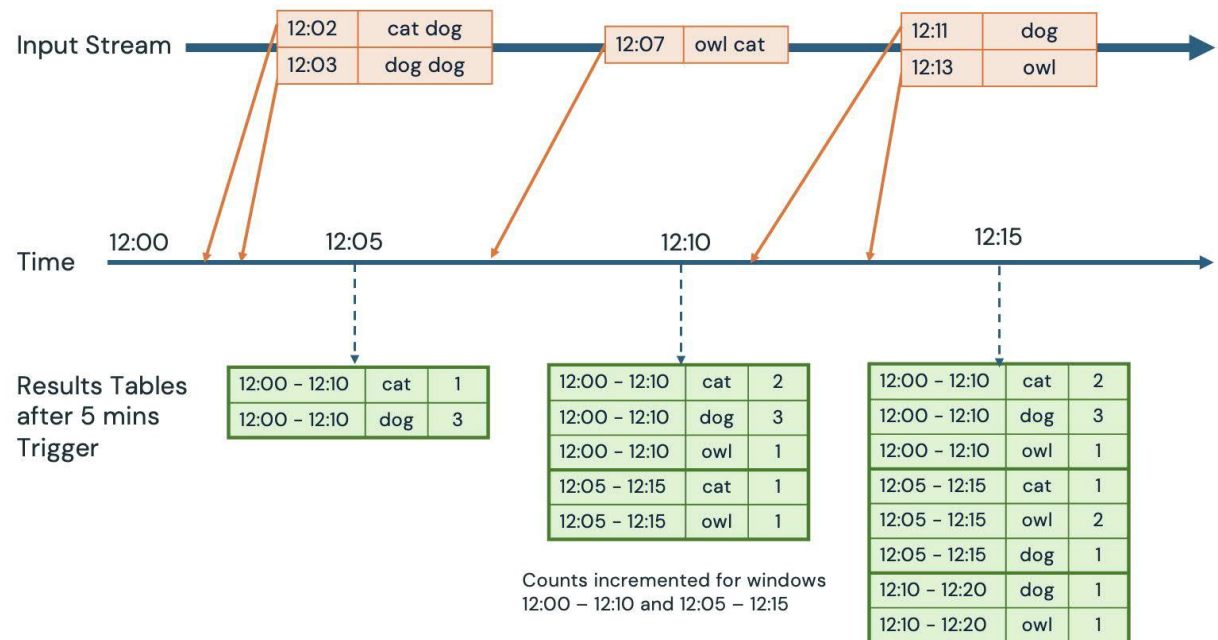


```
# Wait up to 10 minutes for late-arriving data
windowedCounts = df \
    .withWatermark("timestamp", "10 minutes") \
    .groupBy(window("timestamp", "10 minutes")) \
    .count()
```

Time Based Windows

Sliding window example

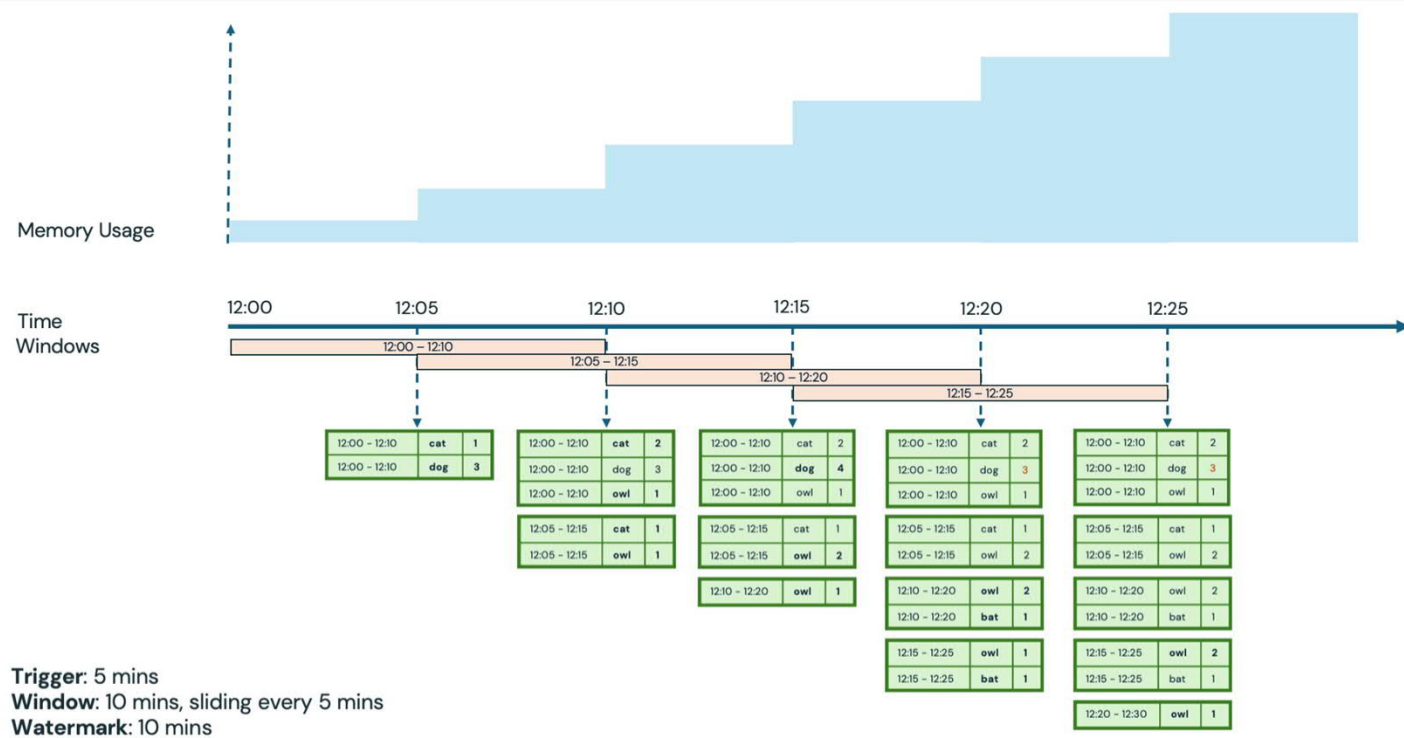
```
windowedDF =  
(  
  eventsDF  
    .groupBy(window("eventTime",  
      "10 minutes",  
      "5 minutes"))  
    .count()  
    .writeStream  
    .trigger(processingTime="5 minutes")  
)
```



Challenges: Memory Pressure

Long running query example

- Size of intermediate state will keep increase over time.
- Default behaviour is to maintain state in executor memory and DBFS.
- GC pauses will take longer and longer. Failure is inevitable.



Solution 1: RocksDB as external state store

One technique to reduce memory usage on the cluster is to save intermediate state off-heap.

```
spark
spark.conf.set(
  "spark.sql.streaming.stateStore.providerClass",
  "com.databricks.sql.streaming.state.RocksDBStateStoreProvider")
```

References:

[Databricks Blog on RocksDB](#)

[Structured Streaming Programming Guide](#)



Solution 2: Watermarking / Late Threshold

Capture “Event Time” on incoming data and set Watermarks.

Purpose:

1. Reclaim memory by purging old state
2. Set expectations on “how late is too late?”

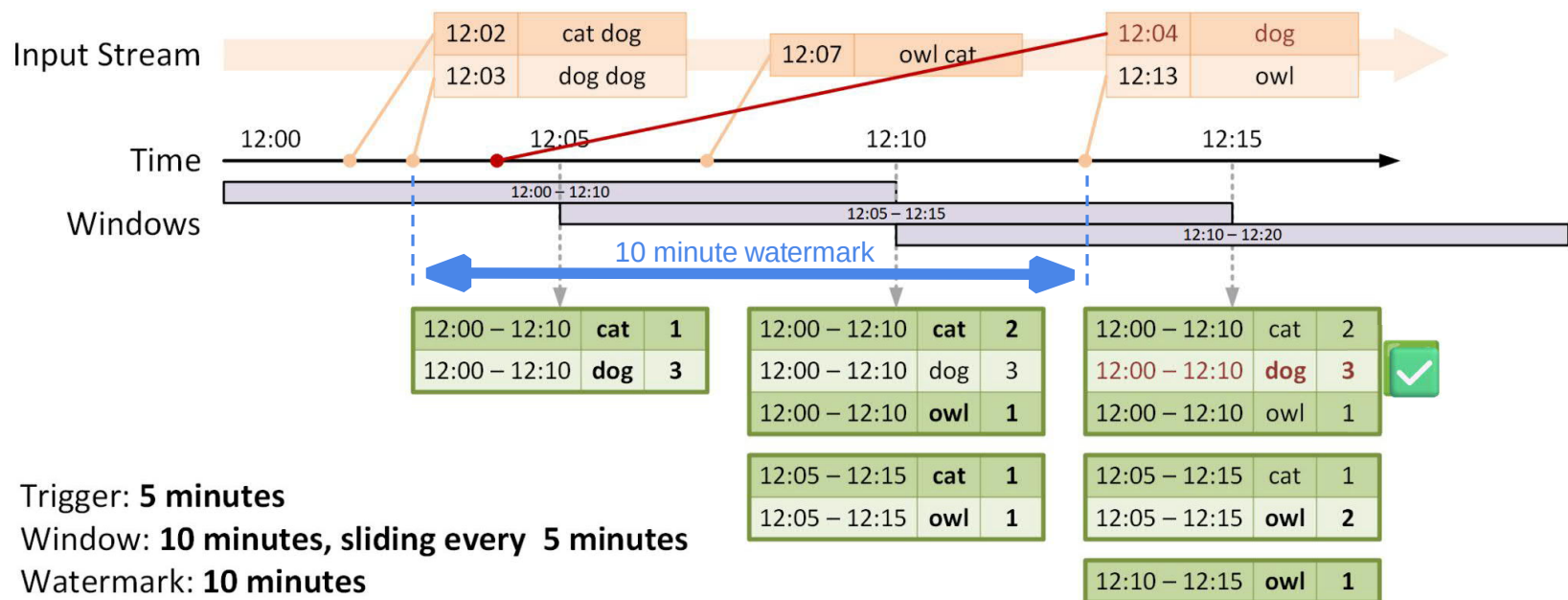
Event Time < (Max event time seen by the engine - late threshold)

```
Spark
.readStream
.withWatermark("event_timestamp", "10 minutes")
.<source info>
```



Handling Late Arrival Events with Watermark

Delayed data will be processed if within Watermark threshold - Guaranteed!

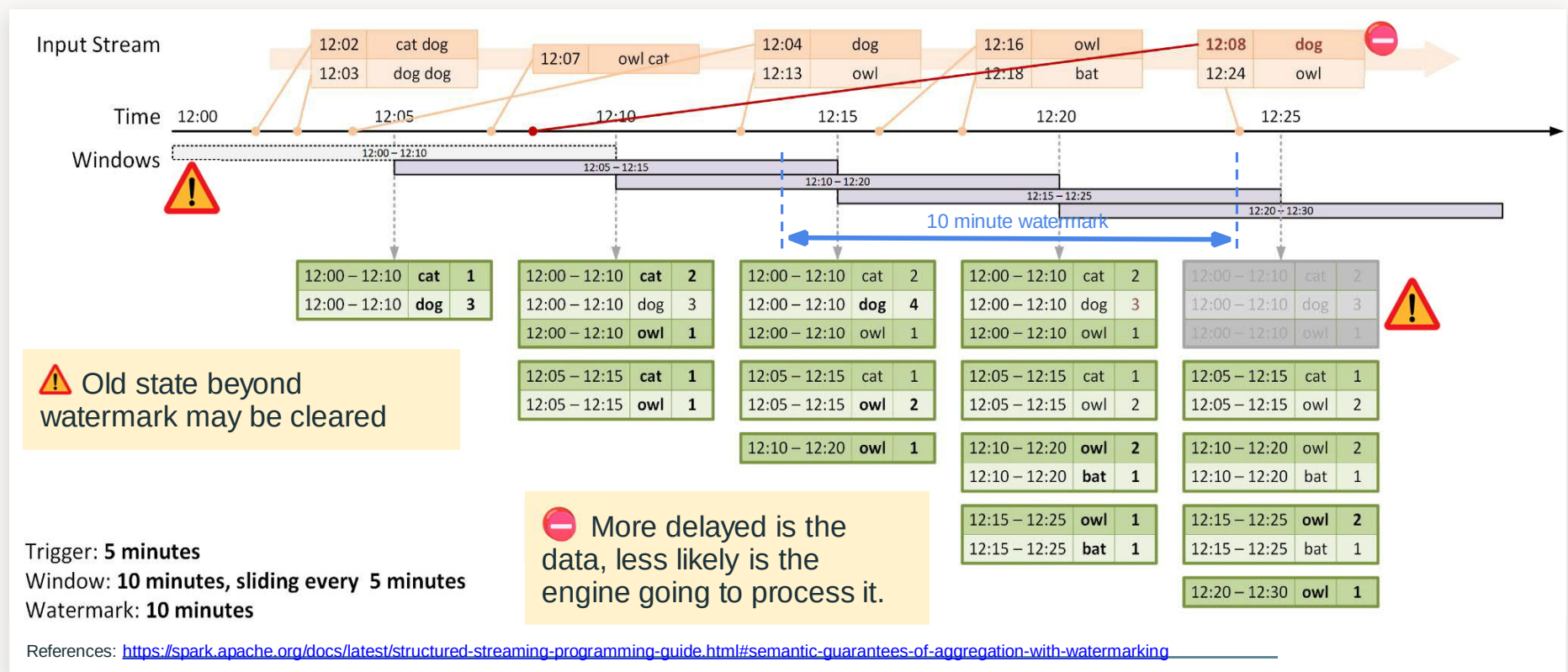


References: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#semantic-guarantees-of-aggregation-with-watermarking>



Control size of state

Late data MAY be dropped





Streaming Joins

Streaming Joins



Join Operations

- Structured Streaming supports joining a streaming Dataset/DataFrame with a static Dataset/DataFrame as well as another streaming Dataset/DataFrame.
- The result of the streaming join is generated incrementally, similar to the results of streaming aggregations in the previous examples.
- In all the supported join types, the result of the join with a streaming Dataset/DataFrame will be the exactly the same as if it was with a static Dataset/DataFrame containing the same data in the stream.



Managing Memory Pressure

As the stream runs, the size of streaming state will keep growing indefinitely as all past input must be saved as any new input can match with any input from the past.

Avoid Unbounded State by:

1. Define Watermark delays on both inputs
2. Add **Event-Time Constraints** on join condition

e.g. `JOIN ON leftTime BETWEEN rightTime AND rightTime + INTERVAL 1 HOUR`

Reference: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#stream-stream-joins>



Streaming Joins Supported

Left Side ↓	Right Side →	
	Static	Stream
Static	✓ All join types supported - Stateless (This is not on streaming join even though it can be present in a streaming query)	✓ Inner Join - Stateless ✓ Right Outer - Stateless ✗ All Other joins
Stream	✓ Inner Join - Stateless ✓ Left Outer - Stateless ✓ Left Semi - Stateless ✗ All Other joins	✓ Inner Join - Stateful. Watermarks Optional ✓ Left Outer - Stateful. Watermark mandatory on right ✓ Left Semi - Stateful. Watermark mandatory on right ✓ Right Outer - Stateful. Watermark mandatory on left ✓ Full Outer - Stateful. Watermark mandatory on at least side (Specify watermark on both sides + event time constraints for full state cleanup)

Reference: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#support-matrix-for-joins-in-streaming-queries>

